



MS SQL Server Interview Series

With Sai Reddy

SQL Interview Answers – Day 2

1. Average Salary per Department

```
SELECT
    e.DepartmentID,
    d.DepartmentName,
    AVG(s.GrossSalary) AS AverageGrossSalary
FROM Salaries s
JOIN Employees e ON s.EmployeeID = e.EmployeeID
JOIN Departments d ON e.DepartmentID = d.DepartmentID
GROUP BY e.DepartmentID, d.DepartmentName;
```

2. Employees Who Joined in the Last Year

```
SELECT
    EmployeeID,
    Name,
    JoinDate
FROM Employees
WHERE JoinDate >= DATEADD(YEAR, -1, GETDATE());
```

3. Department with Highest Total Net Salary Paid in July 2025

```
SELECT TOP 1
    e.DepartmentID,
    d.DepartmentName,
    SUM(s.NetSalary) AS TotalNetSalary
FROM Salaries s
JOIN Employees e ON s.EmployeeID = e.EmployeeID
JOIN Departments d ON e.DepartmentID = d.DepartmentID
WHERE s.Month = '2025-07'
GROUP BY e.DepartmentID, d.DepartmentName
ORDER BY TotalNetSalary DESC;
```

4. Employees with More Than 2 Days of Leave in a Month

```
SELECT
    a.EmployeeID,
    e.Name,
    FORMAT(a.AttendanceDate, 'yyyy-MM') AS LeaveMonth,
    COUNT(*) AS LeaveDays
FROM Attendance a
JOIN Employees e ON a.EmployeeID = e.EmployeeID
WHERE a.Status = 'Leave'
GROUP BY a.EmployeeID, e.Name, FORMAT(a.AttendanceDate, 'yyyy-MM')
HAVING COUNT(*) > 2;
```

5. Employees with No Salary Record in July 2025

```
SELECT
    e.EmployeeID,
    e.Name
FROM Employees e
LEFT JOIN Salaries s
    ON e.EmployeeID = s.EmployeeID AND s.Month = '2025-07'
WHERE s.SalaryID IS NULL;
```

6. Daily Attendance Count for Each Department

```
SELECT
    e.DepartmentID,
    d.DepartmentName,
    a.AttendanceDate,
    COUNT(*) AS PresentCount
FROM Attendance a
JOIN Employees e ON a.EmployeeID = e.EmployeeID
JOIN Departments d ON e.DepartmentID = d.DepartmentID
WHERE a.Status = 'Present'
GROUP BY e.DepartmentID, d.DepartmentName, a.AttendanceDate
ORDER BY a.AttendanceDate, e.DepartmentID;
```

7. Highest Paid Employee per Department

```
WITH RankedSalaries AS (  
    SELECT  
        e.DepartmentID,  
        d.DepartmentName,  
        e.EmployeeID,  
        e.Name,  
        e.Salary,  
        RANK() OVER (PARTITION BY e.DepartmentID ORDER BY e.Salary DESC) AS SalaryRank  
    FROM Employees e  
    JOIN Departments d ON e.DepartmentID = d.DepartmentID  
)  
SELECT  
    DepartmentID,  
    DepartmentName,  
    EmployeeID,  
    Name,  
    Salary  
FROM RankedSalaries  
WHERE SalaryRank = 1;
```

8. Total Tax Deducted per Department in July 2025

```
SELECT  
    e.DepartmentID,  
    d.DepartmentName,  
    SUM(s.TaxDeducted) AS TotalTaxDeducted  
FROM Salaries s  
JOIN Employees e ON s.EmployeeID = e.EmployeeID  
JOIN Departments d ON e.DepartmentID = d.DepartmentID  
WHERE s.Month = '2025-07'  
GROUP BY e.DepartmentID, d.DepartmentName;
```

9. Monthly Absence Report per Employee

```
SELECT  
    a.EmployeeID,  
    e.Name,  
    FORMAT(a.AttendanceDate, 'yyyy-MM') AS Month,  
    COUNT(*) AS AbsentDays  
FROM Attendance a  
JOIN Employees e ON a.EmployeeID = e.EmployeeID  
WHERE a.Status = 'Absent'  
GROUP BY a.EmployeeID, e.Name, FORMAT(a.AttendanceDate, 'yyyy-MM');
```

10. Attendance Percentage for Each Employee in July 2025

```
SELECT
    a.EmployeeID,
    e.Name,
    COUNT(*) AS TotalDays,
    SUM(CASE WHEN a.Status = 'Present' THEN 1 ELSE 0 END) AS DaysPresent,
    CAST(
        100.0 * SUM(CASE WHEN a.Status = 'Present' THEN 1 ELSE 0 END) / COUNT(*)
        AS DECIMAL(5,2)
    ) AS AttendancePercentage
FROM Attendance a
JOIN Employees e ON a.EmployeeID = e.EmployeeID
WHERE FORMAT(a.AttendanceDate, 'yyyy-MM') = '2025-07'
GROUP BY a.EmployeeID, e.Name;
```

SQL Interview Questions – Day 3

Schema

Students

- StudentID (PK)
- FullName
- Email
- EnrollmentDate

Courses

- CourseID (PK)
- CourseName
- Credits

Enrollments

- EnrollmentID (PK)
- StudentID
- CourseID
- Grade (A, B, C, D, F, NULL)

Assessments

- AssessmentID (PK)
- CourseID
- AssessmentType (Quiz, Assignment, Project, FinalExam)
- MaxMarks

Scores

- ScoreID (PK)
- StudentID
- AssessmentID
- MarksObtained

1. Grade Distribution Pivot

Question:

Pivot the grade distribution per course using SQL Server's PIVOT operator.

Show how many students got each grade (A–F) per course.

Expected Output:

- CourseID
- CourseName
- A_Count, B_Count, C_Count, D_Count, F_Count

2. Course Completion Rate

Question:

Write a query to find the percentage of students who completed each course (i.e., have a non-NULL grade). Use COUNT(*) with FILTER logic.

Expected Output:

- CourseID
- CourseName
- CompletionRate (%)

3. Top Scorer per Course using RANK()

Question:

Using RANK(), find the top scorer(s) for each course by total assessment marks.

Expected Output:

- CourseID
- StudentID
- FullName
- TotalMarks
- Rank

4. Flatten Student Assessment Details in JSON Format

Question:

For each student, return their name and a JSON object listing each course and total marks per course using SQL Server's FOR JSON PATH.

Expected Output (example):

- StudentID
- FullName
- CourseScores_JSON (*JSON string with course and marks*)

5. Dynamic Filtering Based on Optional Parameters

Question:

Write a stored procedure-style query that filters students based on optional parameters: @CourseID, @Grade, @EnrollmentYear. Use IS NULL OR = logic.

Expected Output:

- StudentID, FullName, Grade, CourseID, EnrollmentDate

6. Error-Tolerant Parsing with TRY_CAST

Question:

Assume Enrollments.Grade was entered as free text (e.g., "A+", "Pass", "Fail", etc.). Write a query using TRY_CAST or CASE to normalize these into standard grades (A, B, C, F).

Expected Output:

- EnrollmentID
- RawGrade
- NormalizedGrade

7. Course-Wise Average Using GROUPING SETS

Question:

Using GROUPING SETS, compute:

- Average score per course
- Average score per student
- Overall average

Expected Output:

- CourseID
- StudentID
- AverageMarks

8. Students Missing Any Final Exam**Question:**

Find students who have enrolled in a course but haven't attempted the FinalExam assessment for that course.

Expected Output:

- StudentID
- FullName
- CourseID
- CourseName

9. Most Recent Score Entry per Student using CTE**Question:**

Using a CTE, get the most recent score entry for each student.

Expected Output:

- StudentID
- FullName
- AssessmentID
- MarksObtained
- ScoreDate (*assume this column exists*)

10. Detect Students Enrolled in All Available Courses (Relational Division)**Question:**

Write a query to find students who are enrolled in **every course** offered in the system.

Expected Output:

- StudentID
- FullName

